Dock Tools

Ubuntu VM on Hyper-V | Deployment Plan

This document describes how to deploy Dock Tools on a Windows machine running Hyper-V. The recommended approach is to create a dedicated Ubuntu Server VM inside Hyper-V. This avoids Windows-to-Linux path translation issues, gives the Docker daemon a native Linux socket, and provides a clean, production-grade environment.

Prerequisites on the Windows host:

- Hyper-V enabled (Windows 10/11 Pro, Enterprise, or Education; or Windows Server)
- An External Virtual Switch configured in Hyper-V Manager
- Ubuntu Server 24.04 LTS ISO downloaded (ubuntu.com/download/server)
- Internet access from the VM network

Phase 1 - Create the Hyper-V VM

Open Hyper-V Manager and create a new virtual machine with the following settings:

Setting	Recommended Value
Generation	Generation 2 (UEFI) - disable Secure Boot after creation
RAM	2 GB minimum, 4 GB recommended (enable Dynamic Memory)
Storage	40 GB virtual hard disk (VHDX)
Network Adapter	External Virtual Switch (maps to physical NIC)
Boot ISO	Ubuntu Server 24.04 LTS

Note: After creating the VM, go to Settings > Security and uncheck 'Enable Secure Boot' or select the Microsoft UEFI Certificate Authority template, otherwise Ubuntu will fail to boot.

During the Ubuntu installation wizard:

- Choose 'Ubuntu Server (minimized)' or full server install
- Configure a static IP or note the DHCP address for SSH access
- Enable the OpenSSH server option
- Create a non-root user (e.g. docktools) and set a strong password

Phase 2 - Prepare the VM

SSH into the VM from PowerShell on the Windows host:

```
ssh docktools@<vm-ip>
```

Update the system and install Docker:

```
sudo apt-get update && sudo apt-get upgrade -y
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker
sudo apt-get install -y git curl
docker version   # verify Docker is running
```

Phase 3 - Clone and Configure the Project

Create the deployment directory and clone the repository:

```
sudo mkdir -p /opt/dock-tools
sudo chown $USER:$USER /opt/dock-tools
cd /opt/dock-tools
git clone https://github.com/DSerruya/dock_tools.git .
```

Create the environment file from the template and edit it:

```
cp .env.example .env
nano .env
```

Required .env values:

```
# Generate the secret with: openssl rand -hex 32
WEBHOOK_SECRET=<your-generated-secret>

# Absolute path on the VM - NOT a Windows path
HOST_SCRIPTS_DATA_PATH=/opt/dock-tools/scripts-data

DEFAULT_TIMEZONE=UTC
MANAGER_PORT=80
UI_USERNAME=admin
UI_PASSWORD=<strong-password>
```

Note: HOST_SCRIPTS_DATA_PATH must be the real Linux filesystem path on the VM. Docker bind-mounts this directory into each script container. An incorrect path will cause script containers to start with no code inside.

Phase 4 - Build and Start the Stack

```
cd /opt/dock-tools
docker compose up -d --build

# Verify both services are running
docker compose ps

# Tail logs to confirm healthy startup
docker compose logs -f
```

Expected output from 'docker compose ps':

NAME	IMAGE	STATUS	PORTS
manager	dock-tools-manager	Up	3000/tcp (internal)
nginx	nginx:alpine	Up	0.0.0.0:80->80/tcp

Phase 5 - Configure Windows Firewall

Run the following in PowerShell as Administrator on the Windows host to allow traffic to reach the VM:

```
# Allow inbound HTTP
New-NetFirewallRule -DisplayName "Dock Tools HTTP" -Direction Inbound \
    -Protocol TCP -LocalPort 80 -Action Allow

# Allow inbound HTTPS (if using TLS)
New-NetFirewallRule -DisplayName "Dock Tools HTTPS" -Direction Inbound \
    -Protocol TCP -LocalPort 443 -Action Allow
```

Note: Also verify that the VM's ufw (Ubuntu firewall) is not blocking port 80/443. Run: sudo ufw allow 80/tcp && sudo ufw allow 443/tcp on the VM if ufw is active.

Phase 6 - Optional: Enable HTTPS / TLS

Generate a self-signed certificate (or use Let's Encrypt if you have a domain):

```
cd /opt/dock-tools/nginx/certs
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout key.pem -out cert.pem -subj "/CN=dock-tools.local"
```

Update docker-compose.yml to use the TLS nginx config:

```
  nginx:
    volumes:
      - ./nginx/nginx-tls.conf:/etc/nginx/nginx.conf:ro   # use TLS config
      - ./nginx/certs:/etc/nginx/certs:ro
```

Add MANAGER_TLS_PORT=443 to .env, then restart the stack:

```
  docker compose up -d
```

Phase 7 - GitHub Webhook Setup

GitHub must be able to reach the VM to trigger auto-restarts on git push. Choose the option that fits your network:

Option	Use Case	Notes
Port Forward (router)	Static public IP	Forward port 80/443 to VM IP in router settings
Cloudflare Tunnel	No open ports needed	Free tier available, no public IP required
ngrok	Local testing only	Not suitable for production use

Webhook URL format once the host is accessible:

```
  http://<vm-ip-or-domain>/webhook/<script-name>
```

Phase 8 - Persist Across VM Reboots

Enable Docker and the Dock Tools stack to start automatically on boot:

```
  sudo systemctl enable docker

  # Create a systemd service for the compose stack
  sudo nano /etc/systemd/system/dock-tools.service
```

Paste the following into the service file:

```
  [Unit]
  Description=Dock Tools
  After=docker.service
  Requires=docker.service

  [Service]
  WorkingDirectory=/opt/dock-tools
  ExecStart=/usr/bin/docker compose up
  ExecStop=/usr/bin/docker compose down
  Restart=always
  User=docktools

  [Install]
  WantedBy=multi-user.target
```

```
  sudo systemctl daemon-reload
  sudo systemctl enable dock-tools
```

Phase 9 - Verify the Deployment

Check	How to Verify
Web UI accessible	Open http://<vm-ip> in a browser on the Windows host
Manager API healthy	curl http://<vm-ip>/api/scripts (expect JSON response)
Log streaming works	Add a test script, start it, open the Logs tab in the UI
Webhook triggers	Push to a connected GitHub repo, confirm auto-restart in UI
Survives reboot	Reboot the VM; verify stack restarts automatically

Script Management - Checking for Updates

Each script has a built-in update check that fetches the latest commits from its GitHub repository and lets you install the update with a single click - without touching the script's environment variables.

How to use

1. **Open the Edit modal for any script (click the pencil icon on the script card).**
2. **Click 'Check for Updates' in the bottom-left of the modal.**
3. **Dock Tools fetches from the remote and reports how many commits are behind.**
4. **If updates exist, the latest commit message is shown and an 'Install Update' button appears.**
5. **Click 'Install Update' to pull the latest code.**
6. **Persistent scripts restart automatically; scheduled scripts pick up the new code on next run.**

Note: Environment variables are stored in Dock Tools' own config (scripts.json), not inside the cloned repository. They are always preserved when an update is applied - you never need to re-enter them.

State	What you see
Up to date	Green text: 'Already up to date'
Updates available	Yellow text: '<N> new commits: <latest message>' + Install Update button
After install	Green text: 'Update applied and script restarted' (persistent) or 'Update applied' (scheduled)
Not yet cloned	Error: 'Repository not cloned yet. Start the script first.'

Common Hyper-V / Windows Gotchas

Issue	Solution
Docker socket path	Linux VM uses /var/run/docker.sock natively - no Windows path needed
Volume paths	HOST_SCRIPTS_DATA_PATH is the Linux VM path, never a Windows C:\ path
VM networking	Use External Virtual Switch so the VM gets a real LAN IP
Windows firewall	Open ports 80/443 on both Windows host firewall AND VM ufw
Self-update feature	Requires outbound internet from the VM to reach GitHub
Secure Boot	Must be disabled (or set to UEFI CA) for Ubuntu to boot on Gen 2 VM